

Taller: POO y modificadores de acceso en Python

Instrucciones

- Lee cada fragmento, ejecuta mentalmente el código y responde lo que se pide.
- Recuerda: en Python no hay “modificadores” como en Java/C++; se usan convenciones:
 - Público: nombre
 - Protegido (convención): `_nombre`
 - Privado (name mangling): `__nombre` se convierte a `_<Clase>_nombre`
- No edites el código salvo que la pregunta lo solicite.

Parte A. Conceptos y lectura de código

1) Selección múltiple

Dada la clase:

```
class A:  
    x = 1  
    _y = 2  
    __z = 3
```

```
a = A()
```

¿Cuáles de los siguientes nombres existen como atributos accesibles directamente desde `a`?

- ☒ a.x
- ☒ a._y
- ☐ a.__z
- ☒ a._A__z

2) Salida del programa

```
class A:
    def __init__(self):
        self.__secret = 42

a = A()
print(hasattr(a, '__secret'), hasattr(a, '_A__secret'))
```

¿Qué imprime?

— imprime "false" en el primer parentesis y "true" en el segundo

3) Verdadero/Falso (explica por qué)

- a) El prefijo `_` impide el acceso desde fuera de la clase. — verdadero ya que esta limitado a solo a la clase y subclases
- b) El prefijo `__` hace imposible acceder al atributo. — falso ya que este solo hace un name mangling
- c) El name mangling depende del nombre de la clase. — verdadero ya que segun este sera el como se realizara el name mangling

4) Lectura de código

```
class Base:
    def __init__(self):
        self._token = "abc"

class Sub(Base):
    def reveal(self):
        return self._token

print(Sub().reveal())
```

¿Qué se imprime y por qué no hay error de acceso?

— imprime "abc" porque al usar solo un "_" no se restringe su acceso mediante otras subclases

5) Name mangling en herencia

```

class Base:
    def __init__(self):
        self.__v = 1

class Sub(Base):
    def __init__(self):
        super().__init__()
        self.__v = 2
    def show(self):
        return (self.__v, self._Base__v)

print(Sub().show())

```

¿Cuál es la salida?

- la salida sera "2,1" ya que primero se esta llamando a clase "sub" para luego llamar a "show" que le devuelve "__v=2" y luego "_base__v=1" que esta en la clase base

6) Identifica el error

```

class Caja:
    __slots__ = ('x',)

c = Caja()
c.x = 10
c.y = 20

```

¿Qué ocurre y por qué?

- nos da error porque al hacer "c.y=20" y no esya atribuido

7) Rellenar espacios

Completa para que b tenga un atributo “protegido por convención”.

```

class B:
    def __init__(self):
        self _____ = 99

```

Escribe el nombre correcto del atributo.

- ._numero

8) Lectura de métodos “privados”

```
class M:
    def __init__(self):
        self._state = 0

    def _step(self):
        self._state += 1
        return self._state

    def __tick(self):
        return self._step()

m = M()
print(hasattr(m, '_step'), hasattr(m, '__tick'), hasattr(m,
'_M__tick'))
```

¿Qué imprime y por qué?

— este imprime "true, false, true" ya que primero verifica si "_step" esta atribuido y como este tiene solo un "_" entonces da true, "__tick" da false ya que esta protegido por doble "_" entonces no es hasta que ya pone "_m_tick" que vuelve a dar true

9) Acceso a atributos privados

```
class S:
    def __init__(self):
        self.__data = [1, 2]
    def size(self):
        return len(self.__data)

s = S()
# Accede a __data (solo para comprobar), sin modificar el código de la
clase:
# Escribe una línea que obtenga la lista usando name mangling y la
imprima.
```

Escribe la línea solicitada.

— `Print(s._S__data)`

10) Comprensión de dir y mangling

```
class D:
    def __init__(self):
        self.__a = 1
        self._b = 2
        self.c = 3

d = D()
names = [n for n in dir(d) if 'a' in n]
print(names)
```

- ¿Cuál de estos nombres es más probable que aparezca en la lista: `__a`, `_D__a` o `a`?
Explica.

el unico que podria aparecer seria "`_D__a`" ya que no existe "`a`" y al momento de llamar a "`__a`" como esta tiene los dos "_" su nombre cambia debido al name mangling

Parte B. Encapsulación con @property y validación

11) Completar propiedad con validación

Completa para que saldo nunca sea negativo.

```
class Cuenta:
    def __init__(self, saldo):
        self._saldo = 0
        self.saldo = saldo

    @property
    def saldo(self):
        — return self._saldo

    @saldo.setter
    def saldo(self, value):
        # Validar no-negativo
        — if value >= 0:
            self._saldo = value
```

12) Propiedad de solo lectura

Convierte `temperatura_f` en un atributo de solo lectura que se calcula desde `temperatura_c`.

```
class Termometro:
    def __init__(self, temperatura_c):
        self._c = float(temperatura_c)

    # Define aquí la propiedad temperatura_f:  $F = C * 9/5 + 32$ 
```

Escribe la propiedad.

```
    def temperatura_f(self):
        return self._c * 9 / 5 + 32
```

13) Invariante con tipo

Haz que `nombre` sea siempre `str`. Si asignan algo que no sea `str`, lanza `TypeError`.

```
class Usuario:
    def __init__(self, nombre):
        self.nombre = nombre

    # Implementa property para nombre

    def nombre(self):
        return self._nombre
    def nombre(self, value):
        if isinstance(value, str):
            self._nombre = value
        else:
            raise TypeError("El nombre debe ser una cadena (str)")
```

14) Encapsulación de colección

Expón una vista de solo lectura de una lista interna.

```
class Registro:
    def __init__(self):
        self.__items = []

    def add(self, x):
        self.__items.append(x)

    # Crea una propiedad 'items' que retorne una tupla inmutable con el contenido

    def items(self):
        return tuple(self.__items)
```

Parte C. Diseño y refactor

15) Refactor a encapsulación

Refactoriza para evitar acceso directo al atributo y validar que velocidad sea entre 0 y 200.

```
class Motor:
    def __init__(self, velocidad):
        self.velocidad = velocidad # refactor aquí
```

```
def velocidad(self):
    return self._velocidad
```

```
def velocidad(self, value):
    if (0 <= value <= 200):
        self._velocidad = value
```

Escribe la versión con @property.

16) Elección de convención

Explica con tus palabras cuándo usarías `_atributo` frente a `__atributo` en una API pública de una librería.

— en el primer caso de `"_atributo"` sería para organizar los libros mientras que `"__atributo"` sería para los administradores del lugar que deciden que se hace o no con un libro

17) Detección de fuga de encapsulación

¿Qué problema hay aquí?

```
class Buffer:
    def __init__(self, data):
        self._data = list(data)
    def get_data(self):
        return self._data
```

Propón una corrección.

```
def get_data(self):
    return self._data.copy()
```

18) Diseño con herencia y mangling

¿Dónde fallará esto y cómo lo arreglas?

```
class A:
    def __init__(self):
        self.__x = 1
```

```
class B(A):
    def get(self):
        return self.__x
```

— falla en "return self.__x" ya que no puede acceder de esa manera tendría que ser usando "self._A__x"

19) Composición y fachada

Completa para exponer solo un método seguro de un objeto interno.

```
class _Repositorio:
    def __init__(self):
        self._datos = {}
    def guardar(self, k, v):
        self._datos[k] = v
    def _dump(self):
        return dict(self._datos)

class Servicio:
    def __init__(self):
        self.__repo = _Repositorio()

    # Expón un método 'guardar' que delegue en el repositorio,
    # pero NO expongas _dump ni __repo.

    def guardar(self, k, v):
        self.__repo.guardar(k, v)
```

20) Mini-kata

Escribe una clase ContadorSeguro con:

- atributo "protegido" `_n`
 - método `inc()` que suma 1
 - propiedad `n` de solo lectura
 - método "privado" `__log()` que imprima "tick" cuando se incrementa
- Muestra un uso básico con dos incrementos y la lectura final.